

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Northern University of Bangladesh

COMPILER FRONTEND

A Lexer, Parser & AST Generator Built From Scratch in C

A Lab Final Project Report submitted in partial fulfillment
of the requirements for the course

Compiler Design Lab CSE 3272

PREPARED BY

Md Kamruzzaman (ID: 42240301975) —
Lexer Core & Integration

Mubarak Hossain (ID: 42240301995) —
Lexer Support

Abdullah Al Noman (ID: 42240301964) —
Parser Core

Sadit Al Islam (ID: 42240301963) — Parser
Extensions & AST

Soad Mahmud (ID: 42240301962) — AST
Visualization

Asraful Islam (ID: 42240302008) — Error
Handling & QA

SUPERVISED BY

Sharika Khan
Lecturer

Department of Computer Science and
Engineering

SUBMISSION DATE: 21/08/2026

ABSTRACT

This project presents the design and implementation of a compiler frontend for a small imperative expression language, built entirely from scratch in C99 with no external dependencies. The system covers the first two classical phases of compilation — lexical analysis and syntax analysis — and extends beyond the core deliverable into semantic evaluation. A hand-written scanner converts raw source text into a stream of typed tokens while tracking line and column position for diagnostics. A recursive-descent parser then consumes this stream and builds an Abstract Syntax Tree (AST) that correctly encodes operator precedence and associativity for arithmetic expressions, assignment statements, and — as an extension — `if` and `while` control structures with block scoping. Malformed input is never allowed to crash the program: every grammar production has a corresponding error case that reports a precise line/column location and a human-readable message before exiting with a non-zero status. As a stretch goal, a tree-walking interpreter (invoked with the `--eval` flag) evaluates the AST directly, maintaining a runtime symbol table and reporting undeclared-variable references as semantic errors rather than crashes. The project was developed by a team of six members, each owning a distinct module (lexer core, lexer support, parser core, parser/AST extensions, AST visualization, and error handling/QA), coordinated through a shared grammar and function-signature contract agreed on before implementation began. Correctness was validated using an automated regression harness of 16 paired input/expected-output samples covering valid programs, syntax errors, control-flow constructs, and interpreter semantics, all of which pass under `make test`. The resulting artifact is a compact, dependency-free reference implementation that demonstrates the complete pipeline from source text to executable behaviour, and serves as a practical foundation for exploring further compiler phases such as static typing and code generation.

Keywords: Compiler Design, Lexical Analysis, Recursive-Descent Parsing, Abstract Syntax Tree, Tree-Walking Interpreter, Error Recovery, Symbol Table

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to **Sharika Khan**, Lab Instructor, for guidance and feedback throughout the design and implementation of this project, and to the Department of Computer Science and Engineering, Northern University of Bangladesh, for providing the lab facilities and course structure (CSE 3272 — Compiler Design Lab) that made this work possible. We also thank our fellow classmates for their support during integration testing and demo rehearsal.

TABLE OF CONTENTS

Section	Title / Chapter Name	Page
1.0	Introduction	1
1.1	Project Background	1
1.2	Problem Statement	1
1.3	Objectives	2
1.4	Scope and Limitations	2
2.0	Literature Review & Related Work	3
2.1	Theoretical Background	3
2.2	Comparative Evaluation of Existing Approaches	4
3.0	System Design & Methodology	5
3.1	System Architecture & Pipeline Diagram	5
3.2	Hardware / Software Requirements	6
3.3	Grammar, Algorithms & Workflow Design	6
4.0	Implementation & Testing	7
4.1	System Development & Setup	7
4.2	Testing Strategy & Experimental Results	8
5.0	Conclusion & Future Work	9
5.1	Summary of Contributions	9
5.2	Future Scope	9
-	References	10

CHAPTER 1: INTRODUCTION

1.1 Project Background

Every compiler, regardless of the source language or target platform, begins with the same two front-end phases: lexical analysis, which groups raw characters into meaningful tokens, and syntax analysis, which arranges those tokens into a structure — typically an Abstract Syntax Tree (AST) — that reflects the grammar and precedence rules of the language. These phases are foundational to compiler theory and are the basis on which every later phase (semantic analysis, optimization, and code generation) is built. Understanding them in the abstract is straightforward; implementing them correctly, with no crashes on malformed input and precise diagnostics, is a different and more demanding exercise. This project builds that frontend from first principles in C, deliberately avoiding parser-generator tools (such as Lex/Yacc) or external libraries, so that every stage of the pipeline — scanning, tokenizing, precedence-aware parsing, tree construction, and evaluation — is visible and owned by the team.

1.2 Problem Statement

Compiler theory taught in isolation from implementation leaves a gap: students can describe what a recursive-descent parser does but often have not had to handle the many edge cases that separate a toy parser from a robust one — unexpected end-of-file, unmatched parentheses, unrecognized characters, and undeclared-variable references at evaluation time. Existing classroom examples frequently either omit error handling entirely (and crash on bad input) or only illustrate a single phase in isolation. This project addresses that gap by delivering a small but complete pipeline — from source text to an evaluated result — where every grammar production has a defined, non-crashing error path with line/column reporting, and where the design is validated against an automated, repeatable test suite rather than manual inspection alone.

1.3 Objectives

- **Primary Objective:** Design and implement a compiler frontend (lexer, recursive-descent parser, and AST) for a small expression/assignment language with correctly enforced operator precedence and associativity.
- **Secondary Objective 1:** Extend the grammar to support `if` and `while` control structures with block scoping, without weakening the guarantees of the original core grammar.
- **Secondary Objective 2:** Implement a tree-walking interpreter with a runtime symbol table that evaluates the AST, detects undeclared-variable references as semantic errors, and validate the entire pipeline against an automated regression suite of paired input/expected-output samples.

1.4 Scope and Limitations

Scope Lexical analysis (identifiers, integer literals, operators, parentheses, keywords, comparison operators); recursive-descent parsing of arithmetic expressions, assignment statements, and `if/while` blocks; AST construction and tree-printing; line/column-aware error reporting for every grammar production; a tree-walking interpreter with a variable symbol table; and an automated test harness covering 16 valid, broken, and evaluation samples.

Limitations The language deliberately excludes functions and procedures, a static type system, string and array data types, and code generation to machine code or bytecode. The top level accepts exactly one statement (which may itself be an `if/while` whose block holds many statements), preserving the guarantees of the original single-statement core grammar rather than introducing multi-statement programs.

CHAPTER 2: LITERATURE REVIEW

2.1 Theoretical Background

The structure of this project follows the classical front-end model described in compiler-construction literature (e.g., Aho, Lam, Sethi & Ullman's *Compilers: Principles, Techniques, and Tools*, and Nystrom's *Crafting Interpreters*): source text is first reduced to a token stream by a scanner, then structured into a syntax tree by a parser that encodes grammar rules directly as functions. This project uses recursive-descent parsing, where operator precedence is enforced purely by the calling order of parser functions — `parse_factor()` is invoked from `parse_term()`, which is invoked from `parse_expr()` — rather than by a table-driven or generator-produced parser. The AST produced by this stage acts as an intermediate representation that is independent of both the original text and any specific execution strategy, which is what allows the same tree to be either printed (for inspection) or walked directly by a tree-walking interpreter, the simplest of the classical evaluation strategies (as opposed to compiling to bytecode for a virtual machine or to native code).

2.2 Comparative Evaluation of Existing Approaches

Approach	Methodology Used	Key Advantages	Identified Limitations
Parser-generator tools (e.g., Lex/Yacc, ANTLR)	Grammar specification compiled into a table-driven scanner/parser	Fast to bootstrap for large grammars; well-tested core algorithms	Adds a build-time dependency; generated code obscures how precedence and error recovery actually work — poor fit for a learning-focused project
High-level scripting frameworks (e.g., Python's tokenize/ast)	Reuses a mature, host-language-provided lexer/parser/AST toolchain	Very high productivity; rich existing tooling (visitors, transformers)	Hides the implementation entirely inside the host runtime; unsuitable for demonstrating manual token scanning and grammar-to-code mapping
Proposed Project (hand-written, dependency-free C)	Hand-written character-by-character scanner and recursive-descent parser, each grammar rule mapped one-to-one onto a parser function	Every phase is transparent and owned by the team; precise control over error messages and line/column tracking; zero external dependencies	Requires more manual effort to extend the grammar (e.g., adding new operators or statement forms) than a generator-based approach

CHAPTER 3: SYSTEM DESIGN & METHODOLOGY

3.1 System Architecture & Pipeline Diagram

The system is organized as a linear pipeline with a cross-cutting error-handling module that any stage can invoke. A source file is read into memory, tokenized by the lexer, parsed into an AST by the recursive-descent parser, and then either printed (default) or evaluated by the tree-walking interpreter when the `--eval` flag is supplied. Any stage that detects a malformed input reports through the shared error module and the program exits with a non-zero status instead of crashing.

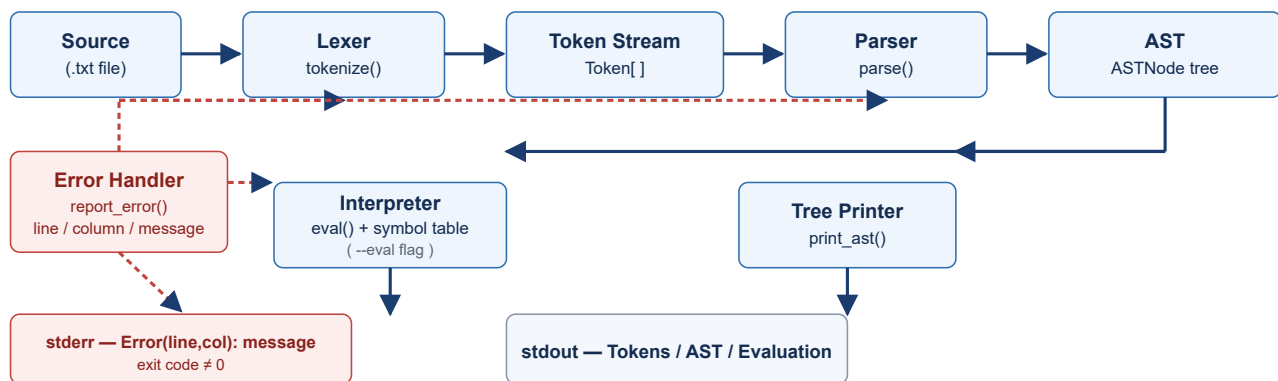


Fig. 3.1 — Compiler frontend pipeline: source → lexer → token stream → parser → AST → tree printer / interpreter, with the error-handling module (dashed) reachable from every stage.

3.2 Hardware / Software Requirements

Category	Item	Specification / Version
Hardware	Development machine	Any standard PC/laptop (Windows, Linux, or macOS) — no special hardware required
Software	Compiler	GCC (or Clang), C99 standard, built with <code>-Wall -Wextra -std=c99</code>
Software	Build tool	GNU Make (mingw32-make on Windows without MSYS2)
Software	Shell / test runner	Bash (Git Bash / MSYS2 / WSL on Windows) for <code>tests/run_tests.sh</code>
Software	Version control	Git
Software	Dependencies	None — standard C library only

3.3 Grammar, Algorithms & Workflow Design

```
statement  → assign_stmt | if_stmt | while_stmt
assign_stmt → IDENTIFIER '=' expr
if_stmt    → 'if' '(' condition ')' block
while_stmt → 'while' '(' condition ')' block
block      → '{' statement* '}'
condition  → expr (('<' | '>' | '<=' | '>=' | '==' | '!=') expr)?
expr       → term (('+' | '-') term)*
term       → factor (('*' | '/') factor)*
factor     → NUMBER | IDENTIFIER | '(' expr ')'
```

Each grammar rule maps directly onto a parser function, so precedence is enforced purely by the call chain `parse_factor() → parse_term() → parse_expr()`: multiplication and division bind tighter than addition and subtraction because `parse_factor` is invoked from inside `parse_term`, which is invoked from inside `parse_expr`. The interpreter walks the resulting AST directly, using an array-based symbol table (variable name → integer value) populated on assignment; looking up a name that has not yet been assigned raises a semantic error reported with line/column rather than an undefined-behaviour crash. Every parser production that can fail (missing `,`, missing identifier, unexpected token/EOF, unrecognized character) has a matching error case wired through `report_error()`.

CHAPTER 4: IMPLEMENTATION & TESTING

4.1 Implementation Workflow

`main.c` wires the full pipeline: it reads the input filename from `argv`, loads the file into a buffer, calls `tokenize()` and prints the `--- Tokens ---` block, checks `had_error()` before proceeding, calls `parse()` and prints the `--- AST ---` block via `print_ast()`, checks `had_error()` again, and — if the `--eval` flag is present — hands the AST to the interpreter, which prints an `--- Evaluation ---` block as each assignment executes. `free_tokens()` and `free_ast()` release all heap memory before exit; the program returns exit code 0 on success and non-zero on any reported error.

Development proceeded through seven milestones (M1–M7): scaffolding and core types, the lexer, the parser and AST, integration and error handling, the test suite, and the two stretch goals — `if/while` control flow and the tree-walking interpreter — each merged only after the previous milestone was fully green. Work was split across six team members, each owning one module (lexer core, lexer support, parser core, parser/AST extensions, AST visualization, and error handling/QA), coordinated by agreeing on shared structs and function signatures before implementation began so that all six could work in parallel without blocking one another.

4.2 Testing Strategy & Experimental Results

Correctness is validated with a regression harness (`make test / tests/run_tests.sh`) that runs every `.txt` file in `tests/samples/` and diffs the actual output against a paired `.expected` transcript. The suite covers 16 samples across four categories: valid arithmetic/assignment programs, syntax-error programs, control-flow (`if/while`) programs (valid and broken), and interpreter-evaluation programs (valid and semantically broken). All 16 samples pass.

Test Case ID	Input Scenario	Expected Output	Observed Output	Status
TC-001	Nested arithmetic: <code>x = 5 + y * (2 - 1)</code>	AST reflects <code>*/</code> binding tighter than <code>+/-</code>	Assign → Identifier: <code>x</code> , BinaryExpr(+) → Number:5, BinaryExpr(*) → ...	PASSED
TC-002	Truncated expression: <code>x = 5 +</code>	Clear syntax error, no crash	Error (line 1, col 8): expected expression after '+', exit ≠ 0	PASSED
TC-003	Malformed if: missing closing paren, <code>if (x < 10 { ... }</code>	Error naming the missing <code>)</code> , no crash	Error (line 1, col 12): expected <code>') '</code>	PASSED
TC-004	Nested control flow with <code>--eval: if (1) { x = 0; while (x < 3) { x = x + 1 } }</code>	Loop executes 3 times; each assignment printed in order	<code>x = 0 / x = 1 / x = 2 / x = 3</code>	PASSED
TC-005	Undeclared variable with <code>--eval: y = x + 1</code>	Semantic error with line/column, not a crash	Error (line 1, col 5): undeclared variable 'x'	PASSED

Beyond the automated suite, the build itself was verified to compile with zero warnings under `-Wall -Wextra -std=c99`, and debug builds were spot-checked for memory leaks around `free_tokens()/free_ast()` during milestone reviews.

CHAPTER 5: CONCLUSION & FUTURE SCOPE

5.1 Summary of Contributions

This project delivered a complete, dependency-free compiler frontend in C: a lexer with accurate line/column tracking, a recursive-descent parser that correctly enforces operator precedence and associativity, an AST with a readable tree-printer, and a crash-free error path for every grammar production. Both stretch goals defined in the original project specification were also completed — `if/while` control structures with block scoping, and a tree-walking interpreter with a symbol table and undeclared-variable detection — meaning all seven planned milestones (M1–M7) were reached, not just the core MVP (M1–M5). The result meets every objective stated in Section 1.3: a working frontend, extended control-flow support, and a validated evaluator, all backed by a 16-sample automated regression suite that runs in seconds via `make test`.

5.2 Future Scope

- Adding function/procedure definitions and calls, requiring a proper call stack and scoping model.
- Introducing a static type system and type-checking pass ahead of evaluation.
- Supporting string and array data types beyond the current integer-only model.
- Compiling the AST to bytecode for a small virtual machine (or to LLVM IR) instead of tree-walking, to explore code-generation and performance trade-offs.
- Adding a REPL mode for interactive experimentation alongside the current file-based interface.

REFERENCES

1. A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2006.
2. R. Nystrom, *Crafting Interpreters*, self-published, 2021. Available: <https://craftinginterpreters.com>
3. Project design specification: `docs/Compiler_design_architecture.md`, this repository.
4. Implementation milestone plan: `docs/MILESTONES.md`, this repository.
5. Project source code repository: https://github.com/zamanrabeen671/compiler_design